SAP SURVIVOR: Guía de Motor Unity (C#)

Este documento contiene la arquitectura completa en C# y la guía de ensamblaje para implementar el juego "SAP SURVIVOR: A Folha Não Pode Parar!" en el motor Unity, creando un juego 2D arcade profesional para escritorio.

1. Configuración del Proyecto

- Abre Unity Hub y crea un nuevo proyecto **2D Core**.
- En la ventana **Hierarchy**, crea los siguientes elementos vacíos (`Create Empty`): **GameManager** y **SpawnManager**.
- Crea los siguientes **Prefabs** arrastrando sprites a la escena y luego a la carpeta de proyecto:
  - **Player:** Sprite con `BoxCollider2D` y `Rigidbody2D` (Gravity Scale = 0). Tag: "Player".
  - **Enemy:** Sprite con `BoxCollider2D` (Is Trigger = true) y `Rigidbody2D` . Tag: "Enemy".
  - **Bullet:** Sprite con `BoxCollider2D` (Is Trigger = true) y `Rigidbody2D` .

2. Scripts C# del Motor del Juego

Crea una carpeta llamada `Scripts` y añade los siguientes archivos. Estos scripts manejan la física, instancias y estado del juego.

PlayerController.cs (Control del Key User)

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    public float speed = 8f;
    public GameObject bulletPrefab;
    public Transform firePoint;
    public float fireRate = 0.2f;
    private float nextFireTime = 0f;
    private Rigidbody2D rb;
    private Vector2 movement;

    void Start() {
        rb = GetComponent<Rigidbody2D>();
    }

    void Update() {
        movement.x = Input.GetAxisRaw("Horizontal");
        if (Input.GetButton("Jump") && Time.time >= nextFireTime) {
            Shoot();
            nextFireTime = Time.time + fireRate;
        }
    }

    void FixedUpdate() {
        rb.MovePosition(rb.position + movement * speed * Time.fixedDeltaTime);
        Vector2 clampedPos = rb.position;
        clampedPos.x = Mathf.Clamp(clampedPos.x, -8f, 8f);
        rb.position = clampedPos;
    }

    void Shoot() {
        Instantiate(bulletPrefab, firePoint.position, firePoint.rotation);
    }
}
```

Bullet.cs (Lógica de los proyectiles)

```
using UnityEngine;

public class Bullet : MonoBehaviour
{
    public float speed = 12f;
    public int damage = 1;

    void Update() {
        transform.Translate(Vector2.up * speed * Time.deltaTime);
        if (transform.position.y > 6f) Destroy(gameObject);
    }

    void OnTriggerEnter2D(Collider2D hitInfo) {
        Enemy enemy = hitInfo.GetComponent<Enemy>();
        if (enemy != null) {
            enemy.TakeDamage(damage);
            Destroy(gameObject);
        }
    }
}
```

Enemy.cs (Bugs y Chamados)

```
using UnityEngine;

public class Enemy : MonoBehaviour
{
    public float speed = 3f;
    public int health = 3;
    public int scoreValue = 100;

    void Update() {
        transform.Translate(Vector2.down * speed * Time.deltaTime);
        if (transform.position.y < -6f) {
            GameManager.Instance.LoseHealth(1);
            Destroy(gameObject);
        }
    }

    public void TakeDamage(int damage) {
        health -= damage;
        if (health <= 0) Die();
    }

    void Die() {
        GameManager.Instance.AddScore(scoreValue);
        Destroy(gameObject);
    }
}
```

SpawnManager.cs (Generador de problemas SAP)

```
using UnityEngine;

public class SpawnManager : MonoBehaviour
{
    public GameObject[] enemyPrefabs;
    public float spawnInterval = 1.5f;
    public float xRange = 8f;

    void Start() {
        InvokeRepeating("SpawnEnemy", 1f, spawnInterval);
    }

    void SpawnEnemy() {
        int index = Random.Range(0, enemyPrefabs.Length);
        Vector2 pos = new Vector2(Random.Range(-xRange, xRange), 6f);
        Instantiate(enemyPrefabs[index], pos, Quaternion.identity);
    }
}
```

GameManager.cs (Controlador principal)

```
using UnityEngine;
using UnityEngine.UI;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;
    public int score = 0;
    public int health = 3;
    public Text scoreText;
    public Text healthText;
    public GameObject gameOverPanel;

    private void Awake() {
        if (Instance == null) Instance = this; else Destroy(gameObject);
    }

    public void AddScore(int amount) {
        score += amount;
        UpdateUI();
    }

    public void LoseHealth(int amount) {
        health -= amount;
        UpdateUI();
        if (health <= 0) GameOver();
    }

    void UpdateUI() {
        if(scoreText != null) scoreText.text = "PONTOS: " + score;
        if(healthText != null) healthText.text = "VIDAS: " + health;
    }

    void GameOver() {
        Time.timeScale = 0f;
        gameOverPanel.SetActive(true);
    }
}
```

3. Conexión en el Editor de Unity

- **El Jugador:** Arrastra `PlayerController` al objeto Player. Crea un objeto vacío hijo llamado "FirePoint" y asígnalo en el inspector del script. Asigna tu prefab `Bullet`.
- **El Generador:** Arrastra `SpawnManager` al objeto vacío de la escena. Aumenta el tamaño de la lista de prefabs y asigna tus distintos enemigos (Chamados, Bugs).
- **La Interfaz:** Crea un Canvas (`UI -> Canvas`), añade textos para puntos y vidas, y un panel de Game Over. Asígnalos al script `GameManager`.